UNIVERSIDAD VERACRUZANA
Facultad de Estadística e Informática
Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

Llamadas a procedimientos remotos (RPC)

Guía de clase · Unidad III

Programación en Red · NRC 19141 · Martes 20 de octubre de 2026

1. ¿Qué es RPC?

Una llamada a procedimiento remoto (RPC, Remote Procedure Call) permite ejecutar una función que vive en otra computadora como si fuera una función local del programa: se le pasan argumentos y se recibe un valor de regreso (López-Fuentes, 2015, sec. 4.5).

Programar directamente con sockets obliga a cada aplicación a resolver a mano el mismo trabajo: diseñar un protocolo de mensajes, codificar y decodificar el texto, delimitar los mensajes (framing), interpretar lo recibido y construir la respuesta. RPC empaqueta todo ese trabajo en un mecanismo reutilizable: la llamada a función.

2. Los dos actores: stub y skeleton

- **Stub (talón), lado cliente:** función local que empaca el nombre de la función y los argumentos, los envía por la red y espera la respuesta.
- **Skeleton (esqueleto), lado servidor:** recibe la petición, desempaca, ejecuta la función de verdad y empaca el resultado de regreso.
- **Marshalling / unmarshalling:** así se llama el empaquetado y desempaquetado de los datos para viajar por la red.

3. Los 10 pasos de una llamada RPC

Cuando el cliente ejecuta saludar("ana"), ocurre lo siguiente:

1. El cliente llama al stub; la llamada parece local.
2. El stub empaca los parámetros (marshalling).
3. El paquete se envía por la red.
4. El sistema operativo del servidor lo recibe.
5. El skeleton desempaca (unmarshalling).
6. Se ejecuta el procedimiento real.
7. El resultado se empaca.
8. Viaja de regreso por la red.
9. El stub desempaca el resultado.
10. El cliente recibe el valor como si la función fuera local.

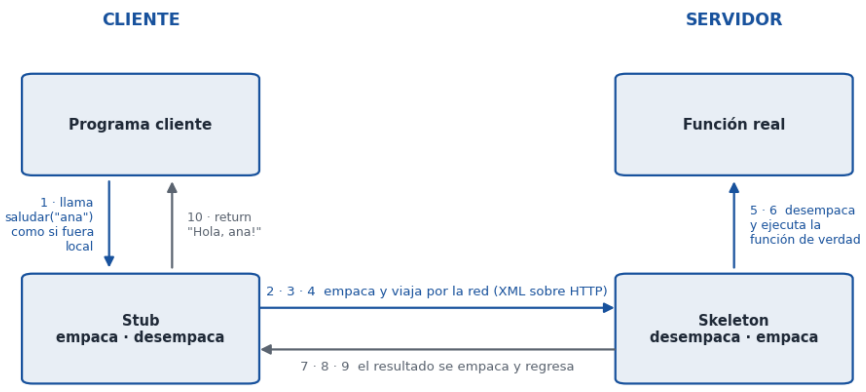


Figura 1. Recorrido de una llamada RPC: el trabajo de red lo hacen el stub y el skeleton.

4. RPC: el ejemplo más claro de middleware

Middleware ("software de en medio") es la capa de software que se coloca entre la aplicación y el sistema operativo con la red, y que ofrece servicios comunes para construir sistemas distribuidos (Coulouris et al., cap. 1).

RPC ilustra la idea mejor que ningún otro mecanismo, por cuatro razones:

- **Está exactamente en medio:** arriba queda la aplicación, abajo los sockets y la red; RPC es la capa intermedia (figura 2).
- **Ofrece un servicio general:** cualquier aplicación puede llamar funciones remotas sin volver a programar protocolos, codificación ni framing.
- **Esconde la complejidad:** toda la maquinaria de comunicación queda reducida a una interfaz simple: llamar una función y recibir un valor.
- **Busca la transparencia de acceso:** su meta es que un recurso remoto se use igual que uno local, sin que el programa distinga la diferencia.

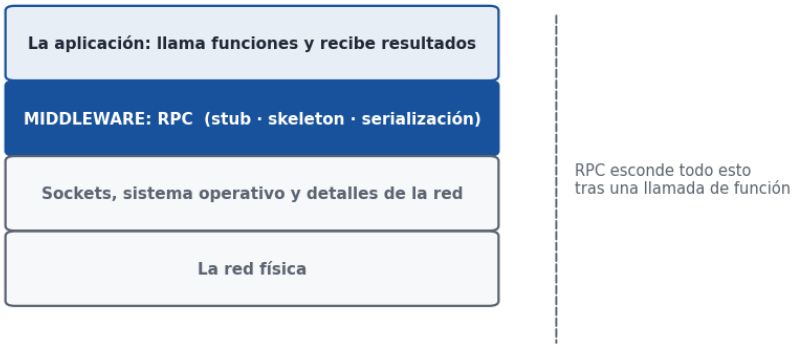


Figura 2. RPC como middleware: una capa entre la aplicación y los detalles de la red.

Transparencia no significa gratuidad: cada llamada remota cruza la red, tarda más que una llamada local y puede fallar por causas ajenas al programa (la red se cae, el servidor no responde). Un buen diseño con RPC toma en cuenta esa latencia y esas posibles fallas.

5. El sobre del mensaje: XML, JSON y HTTP

Para que una llamada RPC viaje, los datos se serializan: se convierten a un formato de texto que ambas máquinas entienden. Los dos formatos más importantes son XML y JSON; el transporte que usa XML-RPC es HTTP.

5.1 XML: texto con etiquetas

XML (eXtensible Markup Language) es un formato de texto que estructura datos con etiquetas anidadas, parecido a HTML pero de propósito general: quien diseña el formato define sus propias etiquetas. Es legible tanto para personas como para máquinas, y muy verboso: cada dato viaja rodeado de etiquetas.

Así se ve, por ejemplo, la llamada saludar("ana") cuando viaja en XML-RPC; la biblioteca genera y lee este sobre por su cuenta:

Ejemplo · una llamada XML-RPC por dentro

```xml
<?xml version="1.0"?>
<methodCall>
  <methodName>saludar</methodName>
  <params>
    <param>
      <value><string>ana</string></value>
    </param>
  </params>
</methodCall>
```

XML aparece en más contextos de los que parece:

- Documentos de Office: los archivos .docx y .xlsx son, por dentro, paquetes de XML comprimido.
- Facturación electrónica en México: el CFDI del SAT es un documento XML con estructura oficial.
- Configuración de software: el pom.xml de Maven, el AndroidManifest.xml y los layouts de pantalla en Android.
- Gráficos vectoriales SVG y mapas de sitio web (sitemaps).
- Feeds de noticias RSS/Atom y servicios web SOAP, además del propio XML-RPC.

5.2 JSON: texto con pares clave-valor

JSON (JavaScript Object Notation) cumple la misma función con una sintaxis mucho más ligera: pares "clave": valor separados por comas. Maneja seis tipos de datos: texto, número, booleano, null, listas y objetos.

Ejemplo · los mismos datos en JSON

```json
{
  "apodo": "ana",
  "turno": 7,
  "van": 12,
  "prioritario": false,
  "servicios": ["turno", "conteo"]
}
```

La comparación directa con el ejemplo anterior resume la diferencia: XML repite etiquetas de apertura y cierre por cada dato; JSON dice cada cosa una sola vez. XML es más formal y permite validar documentos contra esquemas oficiales; JSON es más compacto, más rápido de leer y se convirtió en el formato dominante para intercambio de datos entre aplicaciones.

Contextos donde JSON es el estándar de facto:

- APIs web y servicios REST: las respuestas de servicios de mapas, clima, pagos y redes sociales viajan en JSON.
- Bases de datos NoSQL como MongoDB, que guardan documentos JSON.
- Archivos de configuración modernos, como package.json en proyectos de Node.
- Mensajería entre microservicios y sistemas distribuidos.

En Python, JSON se trabaja con json.dumps() para serializar y json.loads() para reconstruir; es el formato de la Práctica 7 de esta unidad.

5.3 HTTP: el camión que transporta

HTTP es el protocolo de petición y respuesta de la web: un cliente pide, un servidor contesta. XML-RPC lo usa como transporte: cada llamada a función viaja como una petición HTTP que lleva el XML en su cuerpo. Por eso la dirección del servidor empieza con http://. HTTP se estudia a fondo en la Unidad IV.

6. Hola mundo RPC con XML-RPC

XML-RPC viene incluido en la biblioteca estándar de Python: no se instala nada. El servidor registra funciones con un nombre público; el cliente crea un proxy y las llama como si fueran suyas.

Código 1 · servidor_rpc.py

```python
from xmlrpc.server import SimpleXMLRPCServer

def saludar(apodo):
    print("Me llamaron con:", apodo)
    return "Hola, " + apodo + "!"

servidor = SimpleXMLRPCServer(("localhost", 5000))
```

```
servidor.register_function(saludar, "saludar")
print("Servidor RPC en el puerto 5000...")
servidor.serve_forever()
```

Código 2 · cliente_rpc.py

```
import xmlrpc.client

servidor = xmlrpc.client.ServerProxy("http://localhost:5000")
respuesta = servidor.saludar("ana")
print(respuesta)
```

Puntos clave

- register_function expone la función con un nombre público; serve_forever deja al servidor atendiendo llamadas.
- ServerProxy es el stub: servidor.saludar("ana") empaca los datos, los envía y espera la respuesta, todo de forma invisible.
- La URL http://localhost:5000 delata el transporte: la llamada viaja como una petición HTTP con XML adentro.
- La evidencia de que el código corrió del otro lado: el mensaje "Me llamaron con: ana" aparece en la terminal del servidor, no en la del cliente.
- No hay socket, encode ni framing a la vista: el middleware los ejecuta por debajo. Siguen existiendo; solo dejaron de ser responsabilidad del programador.

7. Ejercicio de clase: LA FILA versión RPC

Reimplemente el sistema de turnos LA FILA con XML-RPC, en parejas, sin escribir ni una línea de sockets:

- **El servidor registra dos funciones:** tomar_turno(apodo), que devuelve el número de turno (consecutivo, sin repeticiones, protegido con threading.Lock), y cuantos_van(), que devuelve el total de turnos asignados.
- El cliente pide el apodo por teclado, llama ambas funciones y muestra los resultados.
- Prueben con tres o cuatro clientes al mismo tiempo contra el mismo servidor y verifiquen que los turnos salgan consecutivos.

Discusión de cierre

- ¿Qué pasó con el servicio UDP de CUANTOS? (Ya no hace falta: por debajo, todo viaja por TCP/HTTP.)
- ¿Qué se ganó y qué se perdió respecto a la versión programada con sockets?
- Llamen desde el cliente una función que no existe: ¿qué excepción llega y dónde se generó?

Entregable en clase: la_fila_rpc_servidor.py y la_fila_rpc_cliente.py funcionando en la demostración final.

Referencias

- López-Fuentes, F. (2015). Sistemas Distribuidos. UAM — sección 4.5: modelos de comunicación, paso de mensajes y RPC.
- Tanenbaum, A. S. & Van Steen, M. Sistemas Distribuidos: Principios y Paradigmas — pasos de una llamada RPC, stub, skeleton y marshalling.
- Coulouris, G., Dollimore, J., Kindberg, T. & Blair, G. Distributed Systems: Concepts and Design — capítulo 1 (middleware y transparencia) y capítulo 4 (comunicación entre procesos).
- Documentación oficial de Python: módulos xmlrpc.server y xmlrpc.client.